Part 1: Conceptual Questions (Testing Core Concepts)

1. **What is the DOM (Document Object Model)?** Explain how JavaScript interacts with HTML using the DOM.
2. **Selector Differences:** Explain the difference between `document.getElementById()`, `document.getElementsByClassName()`, and `document.querySelector()`. Which one is the most flexible and why?
3. **Event Listeners:** What is an event listener in JavaScript? List three common DOM events (excluding `"click"`) and describe what triggers them.
4. **Input Values:** How do you retrieve the text that a user has typed into an HTML `<input>` element using JavaScript?
5. **Text vs. HTML:** What is the difference between changing an element's content using `.textContent` versus `.innerHTML`? What is a potential security risk of using `.innerHTML`?

Part 2: Practical Coding Challenges (Hands-on HTML + JS)

For these questions, assume you are writing the JavaScript code to interact with a basic HTML page.

6. **The Basic Alert:** Write the HTML for a button with the text "Greet". Write the JavaScript event listener that triggers a browser `alert("Hello World!")` when this button is clicked.
7. **Heading Text Changer:** You have an HTML heading: `<h1 id="main-title">Welcome</h1>` and a button. Write the JavaScript code so that clicking the button changes the text of the heading to `"JavaScript is Awesome!"`.
8. **Dynamic Input Display:** Create an input text field, a button, and an empty paragraph `<p id="display"></p>`. Write the JavaScript code so that when the button is clicked, the text typed into the input field appears inside the paragraph.
9. **Random Color Changer:** Write a JavaScript function that triggers on a button click and changes the `document.body.style.backgroundColor` to a random color chosen from this array: `const colors = ['red', 'blue', 'green', 'yellow', 'purple', 'orange'];`
10. **Show/Hide Element:** You have a paragraph of text `<p id="secret-text">This is a secret!</p>` and a button. Write the JavaScript code to

toggle the visibility of the paragraph (hide it if it's showing, show it if it's hidden) when the button is clicked.

11. **Live Character Counter:** Create a `<textarea id="user-message">` and a paragraph `<p id="char-count">0 characters</p>`. Write JavaScript that updates the character count in real-time as the user types into the textarea. (Hint: Use the `"input"` event).
12. **Simple Addition Calculator:** Create two number inputs, a button, and a result paragraph. Write the JavaScript to add the two numbers together when the button is clicked and display the sum in the paragraph. *(Warning: Make sure the numbers don't concatenate as strings!)*
13. **Image Switcher:** You have an image tag `<img id="gallery" src="dog.jpg">` and a button. Write the JavaScript code to change the `src` attribute of the image to `"cat.jpg"` when the button is clicked.
14. **Hover Highlight:** Create an HTML list item `<li>Item 1</li>`. Write JavaScript event listeners so that when the user's mouse hovers over the list item, its text color changes to orange, and when the mouse leaves, it changes back to black.
15. **Disable Button Validation:** Create a text input and a submit button. Write JavaScript that keeps the submit button disabled (`button.disabled = true`) until the user has typed at least 5 characters into the input field.

Roadmap of the essential JavaScript concepts

1. Essential ES6+ Syntax (The Daily Drivers)

These are modern JavaScript syntax features that you will write in almost every single React component.

- **`let` and `const`**: Understand block scoping. In React, you will almost always use `const` because React state updates are handled immutability (you don't reassign variables directly).
- **Arrow Functions (`() => {}`)**: Used for defining functional components, writing event handlers, and passing inline functions.
    - *Key detail:* Understand the implicit return syntax: `const add = (a, b) => a + b;`
- **Template Literals (Backticks `` ` ``)**: Used for dynamic string interpolation, especially when dynamically assigning CSS class names.
    - *Example:*`className={`btn btn-${status}`}`

---

2. Destructuring, Spread, and Rest (Data Handling)

React passes data around in objects (Props) and arrays (State). You must be comfortable manipulating them.

- **Property & Array Destructuring**:
    - *Why:* React components receive parameters as a single `props` object. Destructuring lets you pull out exactly what you need.
    - *Example:*`const { name, age } = user;` or `const [count, setCount] = useState(0);`

- **Spread Operator (`...`)**: Used to copy, merge, or update arrays and objects without mutating the original data (immutability is a core React rule).
    - *Example:* `const updatedUser = { ...user, age: 26 };`
- **Rest Parameter (`...`)**: Used to collect remaining properties.
    - *Example:* `const { admin, ...otherProps } = user;`

---

3. Array Methods (Crucial for React UI)

In React, you don't use `for` loops to render lists of items. Instead, you use functional array methods.

- **`.map()`**: **The most important array method.** It transforms an array of data into an array of React elements.
- **`.filter()`**: Used to remove items from a list (e.g., deleting a todo item from state).
- **`.reduce()`**: Useful for aggregating data (e.g., calculating a shopping cart total).
- **`.find()` and `.includes()`**: Used for searching and conditional checks.

---

4. Conditionals & Logical Operators (For JSX)

Since React uses JSX (HTML-like syntax inside JS), you cannot use standard `if/else` statements inside your UI markup. You must use expression-based conditionals.

- **Ternary Operator (`condition ? true : false`)**: Used for "either/or" conditional rendering in JSX.
- **Logical AND (`&&`)**: Used for short-circuit evaluation (rendering something *only* if a condition is true).
    - *Example:* `{isLoggedIn && <Dashboard />}`
- **Optional Chaining (`?.`)**: Safely accessing deeply nested object properties without throwing an error if a parent property is `null` or `undefined`.
    - *Example:* `user?.profile?.avatar`
- **Nullish Coalescing (`??`)**: Providing fallback values.
    - *Example:* `const performance = rating ?? "N/A";`

---

5. ES Modules (`import` and `export`)

React applications are built out of modular files. You must understand how to share code between files.

- **Named Exports**: `export const MyComponent = ...` $\rightarrow$ Imported as `import { MyComponent } from './file'`
- **Default Exports**: `export default MyComponent` $\rightarrow$ Imported as `import MyComponent from './file'`

---

6. Asynchronous JavaScript (Fetching Data)

Most React apps interact with external APIs. You need to know how to handle asynchronous operations.

- **Promises**: Understanding `.then()` and `.catch()`.
- **`async` / `await`**: The modern, cleaner way to handle promises. You will use this inside React's `useEffect` hook or event handlers to fetch data.
- **`fetch()` API (or Axios)**: Knowing how to make `GET` and `POST` requests.